



**Кафедра информационных систем
в химической технологии**

**Институт тонких химических технологий
им. М.В. Ломоносова
РТУ МИРЭА**



ПРОГРАММИРОВАНИЕ В ЗАДАЧАХ ХТ

Лекция 8. Управление проектами и КОДОМ

Содержание лекции

В данной лекции будут рассмотрены следующие темы:

-  Стандартные и подключаемые библиотеки Python
-  Пакетные менеджеры
-  Системы управления версиями
-  Применение Git и командная разработка

Стандартные библиотеки Python

- ☎ Многие десятки модулей, реализующих востребованные функции, распространяемые вместе с дистрибутивом Python
 - ☉ Часть модулей написаны на С и обеспечивают доступ к системным функциям, например, ввод/вывод из файлов
- ☎ Модули стандартных библиотек, как правило, не требуют отдельной установки, но перед использованием в коде должны быть импортированы
 - ☉ Инсталлятор Python для MS Windows устанавливает по умолчанию все модули стандартных библиотек
 - ☉ В Unix-подобных ОС Python устанавливается в виде набора пакетов, что может потребовать использования дополнительных команд пакетного менеджера ОС для установки модулей стандартных библиотек

Дополнительные библиотеки Python

- 📡 Разрабатываются и поддерживаются сторонними компаниями и Open Source сообществом
- 📡 Насчитывают сотни тысяч компонент
 - 🕒 От простых программ и модулей до пакетов и сложных каркасов для разработки приложений
- 📡 Примеры: NumPy, SciPy, matplotlib, pandas и т.п.
- 📡 Требуют отдельной установки
 - 🕒 Обычно установка выполняется пакетным менеджером Python
 - Пакеты устанавливаются из индексов (репозиториев) пакетов

Менеджеры пакетов Python

pip

- ◉ Стандартный менеджер пакетов, встроен в Python 3.4+
- ◉ Устанавливает пакеты из PyPI и других репозиториев
- ◉ Не имеет встроенной поддержки изоляции окружений

conda

- ◉ Устанавливается посредством Anaconda или Miniconda
- ◉ Уклон в пакеты и среды разработки для науки, Data Science и AI/ML
- ◉ Устанавливает пакеты из Conda-Forge
- ◉ Дистрибутив Anaconda может использоваться без подключения к Интернет

poetry, pdm

- ◉ Устанавливаются посредством pip
- ◉ Обеспечивают изоляцию проектов
- ◉ Современный подход к управлению зависимостями

Индексы пакетов Python

 Позволяют скачивать и публиковать пакеты

 PyPI

- ⦿ Официальный и самый популярный репозиторий Python-пакетов
- ⦿ Используется по умолчанию pip и другими менеджерами
- ⦿ Содержит более 500 000 пакетов
- ⦿ Открытый и бесплатный
- ⦿ Нет строгой модерации – может содержать пакеты с вредоносным кодом

 Conda-Forge

 Anaconda Cloud

 DevPI

 Google Artifact Registry

 AWS CodeArtifact

Основные команды pip

- 📡 Команда `list` выводит список установленных пакетов с указанием их версий

```
pip list
```

- 📡 Команда `install` устанавливает указанный пакет и все его зависимости (пакеты, нужные ему для работы)

```
pip install SomePackage
```

- 📡 Команда `install -U` обновляет указанный пакет до последней версии и все его зависимости до поддерживаемых версий

```
pip install -U SomePackage
```

- 📡 Команда `delete` удаляет указанный пакет

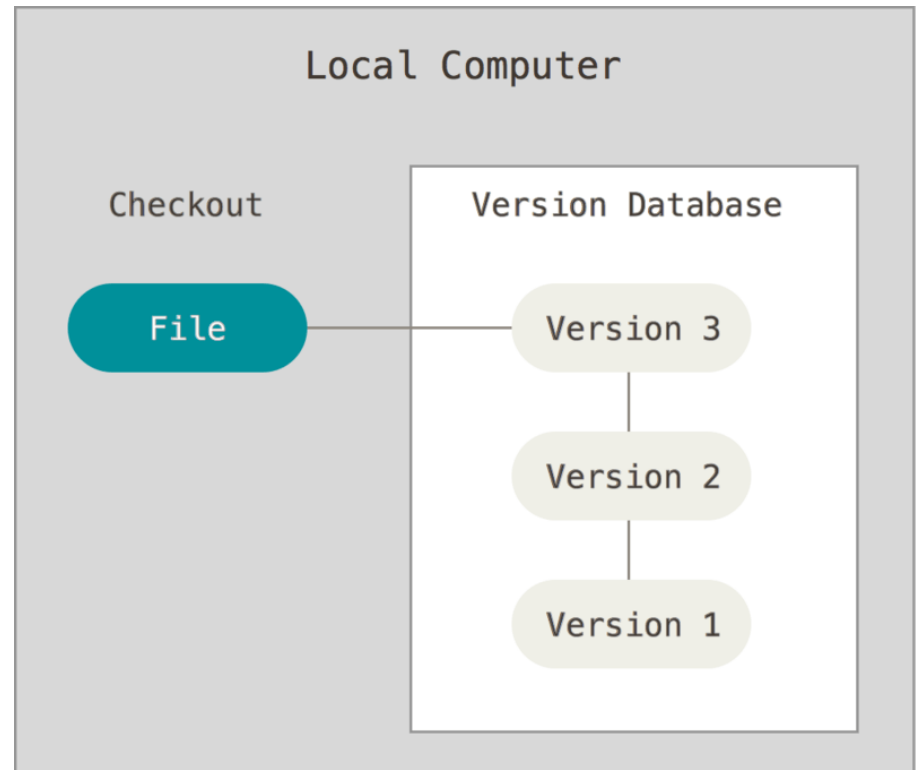
```
pip delete SomePackage
```

Управление версиями

- 🌀 Version Control System (VCS) – специализированное ПО, позволяющее хранить несколько состояний (версий) одного файла
- 🌀 Без VCS: сохранение изменений в файле приводит к безвозвратной потере его предшествующего состояния
 - 🕒 В случае сохранения нежелательных изменений «откатить» их не получится
- 🌀 При использовании VCS: каждое сохранение файла порождает новую версию, хранящуюся наравне с ранее созданным
 - 🕒 В случае сохранения нежелательных изменений можно вернуться к состоянию файла на момент предыдущего сохранения
 - 🕒 Можно перейти к любой ранее созданной версии
 - 🕒 Можно сравнивать версии файлов
 - 🕒 Можно поддерживать существование альтернативных версий

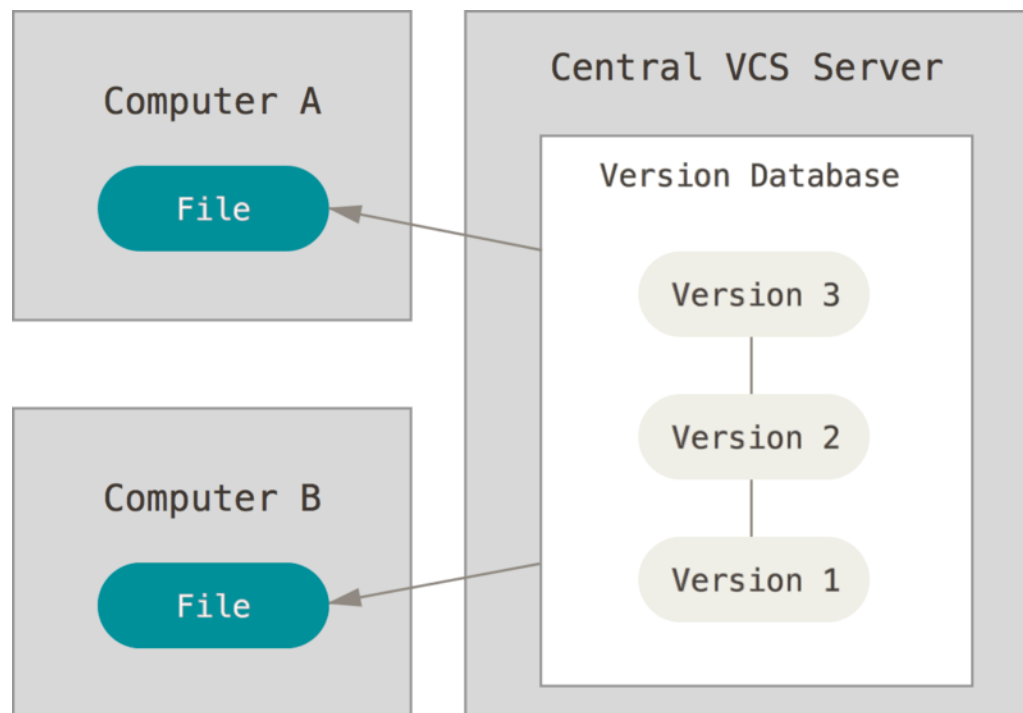
Локальная CVS

- Позволяет «откатить» изменения
- Пример:
 - Revision Control System (разработана в 1980-х)
- Нет поддержки командной разработки
- Версии хранятся в одном месте
 - Угроза потери данных без организации резервного копирования



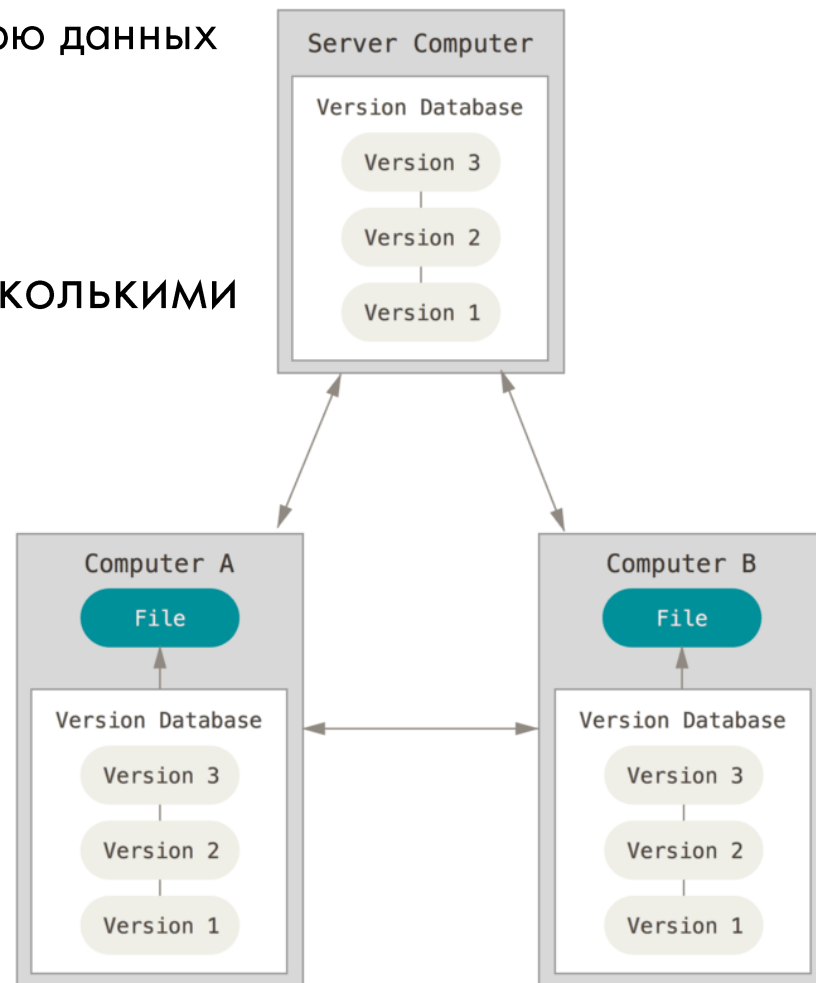
Централизованная CVS

- Поддерживает командную разработку
- Примеры:
 - CVS, Subversion, Perforce
- Сервер CVS является единой точкой отказа
- Версии хранятся в одном месте
 - Угроза потери данных без организации резервного копирования



Распределенная CVS

- Данные версий продублированы в нескольких системах
 - Отказ сервера CVS не влечет потерю данных
- Примеры:
 - Git, Mercurial, Bazaar, Darcs
- Можно взаимодействовать с несколькими удаленными репозиториями
 - Участие в нескольких проектах



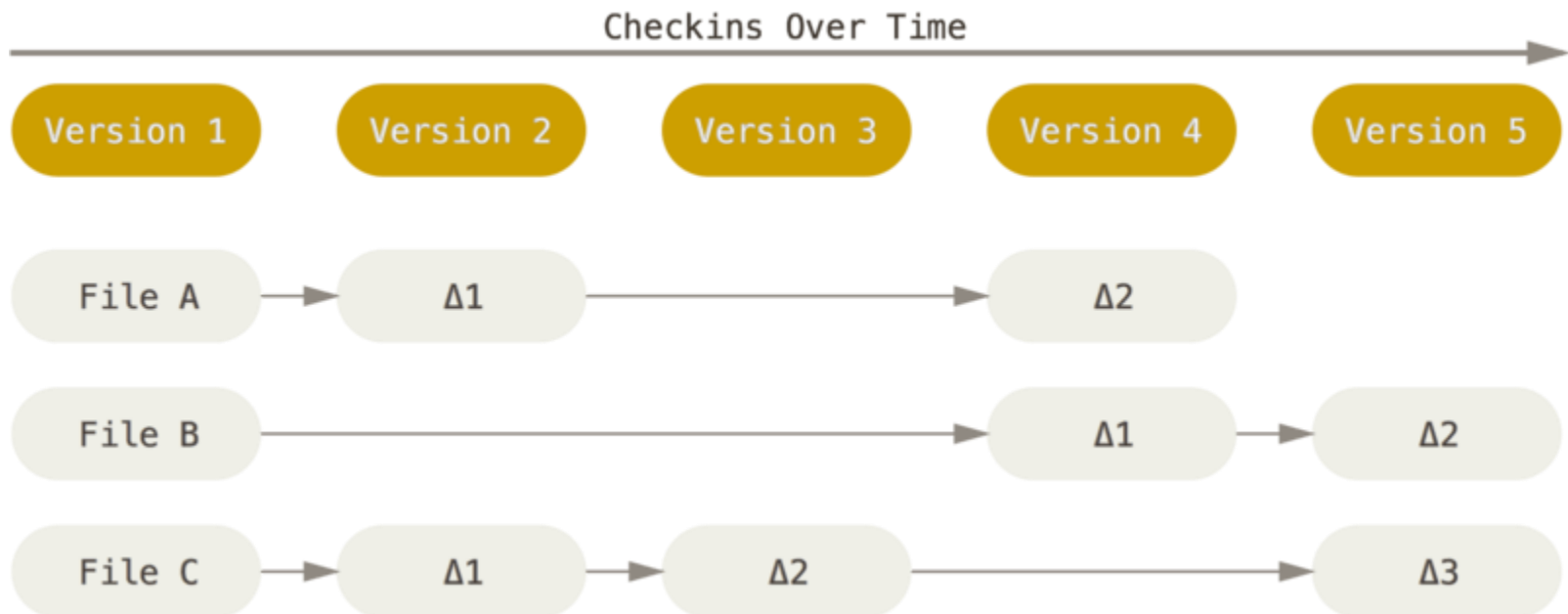
Git

- 🌐 Создан в 2005 году сообществом разработчиков ядра Linux, как ответ на потребность в бесплатной VCS для управления кодом проекта
 - 🌐 Линус Торвальдс – один из создателей Git
- 🌐 Главные характеристики:
 - 🌐 Скорость
 - 🌐 Простая архитектура
 - 🌐 Хорошая поддержка нелинейной разработки
 - 🌐 Полная децентрализация
 - 🌐 Эффективное управление большими проектами



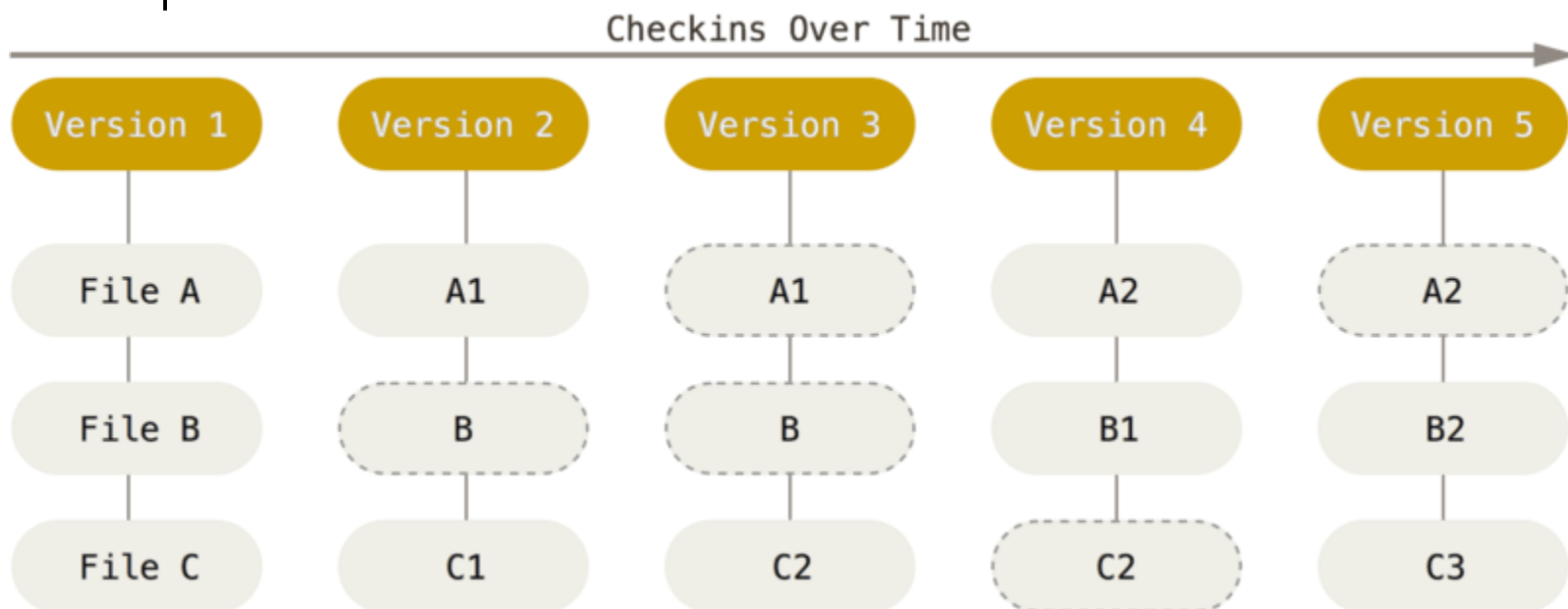
Контроль версий, основанный на различиях

- Хранение исходных файлов и изменений, сделанных в каждом файле со временем
- Используется CVS, Subversion, Perforce, Bazaar и др.
- Git так не работает



Контроль версий, основанный на снимках

- 🌐 Git в момент создания новой версии группы файлов сохраняет состояние каждого файла (снимок) и сохраняет ссылку на этот снимок
 - ⦿ Если файл с момента создания предыдущей версии не был изменен, его новый снимок не создается, а создается новая ссылка на его ранее сохраненный снимок

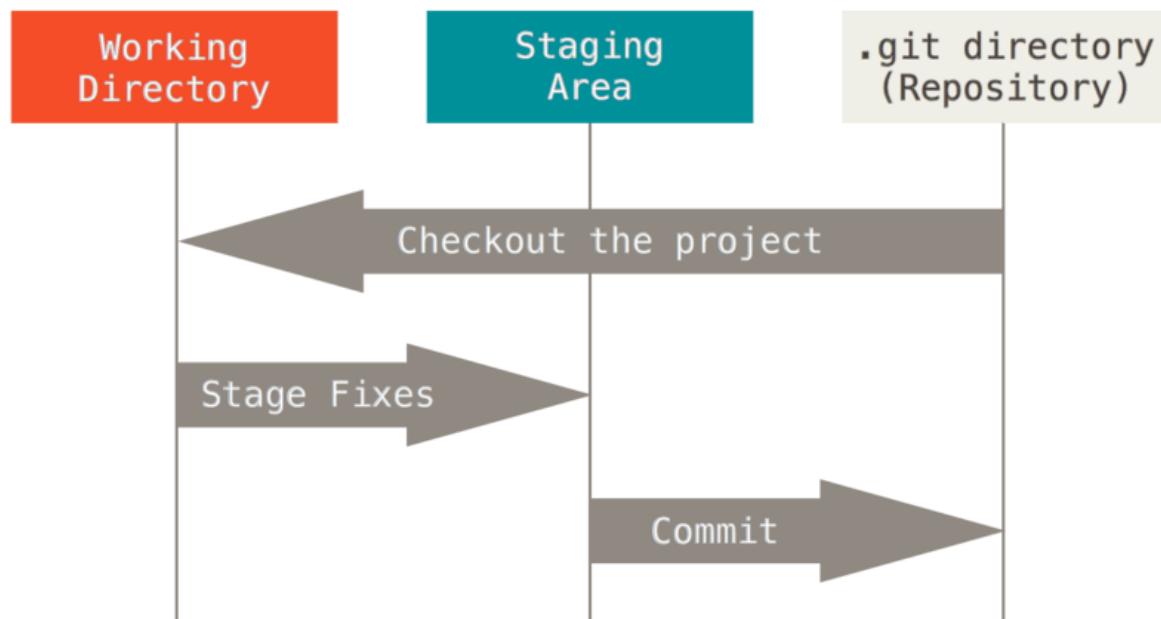


Особенности Git

- 📶 Подавляющее большинство операций выполняется локально
 - ⦿ Быстрота (нет задержек, связанных с обращением к серверу)
 - ⦿ Можно продолжать работать над кодом без соединения с сервером
- 📶 Целостность данных поддерживается на уровне архитектуры
 - ⦿ При сохранении в Git для любых данных для них вычисляется хеш-сумма. Обращение к данным затем осуществляется по этой хеш-сумме
 - ⦿ Любое изменение файлов влечет изменение значения их хеш-суммы и всегда отслеживается Git
 - ⦿ Git использует алгоритм хеширования SHA-1
- 📶 Подавляющее большинство операций добавляют данные в базу Git, данные стираются только особыми командами
 - ⦿ Сложно случайно удалить данные
 - ⦿ При экспериментах с системой данные защищены от потери

Основные состояния файлов в Git

- 📶 Modified (Изменен) – изменен по сравнению с предыдущей зафиксированной версией
- 📶 Staged (Подготовлен) – отмечен для включения в следующую версию
- 📶 Committed (Зафиксирован) – сохранен в локальной базе Git как часть последней зафиксированной версии



Основные директории Git

- 📁 Git-директория – директория для размещения метаданных и базы объектов проекта
 - 🕒 Копируется при клонировании репозитория
- 📁 Область подготовленных файлов – файл, расположенный в Git-директории, где хранится информация о файлах, которые попадут в следующий коммит (зафиксированную версию)
- 📁 Рабочая директория – директория для размещения файлов в их естественном виде. Здесь они доступны для редактирования.
 - 🕒 Директория куда распаковываются файлы из сжатой базы данных Git-директории после клонирования проекта

Установка Git

 Git доступен для всех основных операционных систем

- ⦿ Linux
- ⦿ Windows
- ⦿ Mac

 Установка в Linux

- ⦿ Fedora (RHEL, CentOS):
`sudo dnf install git-all`
- ⦿ Debian (Ubuntu):
`sudo apt install git`

 Установка в Windows выполняется при помощи GUI

 Узнать установленную версию

- ⦿ `git --version`

Конфигурация Git

Глобальный уровень

- ⦿ Для всех пользователей и всех репозиториев
- ⦿ Место хранения: `/etc/gitconfig`
- ⦿ Команда: `git config --system`

Уровень пользователя

- ⦿ Для всех репозиториев пользователя
- ⦿ Место хранения: `~/.gitconfig`
- ⦿ Команда: `git config --global`

Уровень репозитория

- ⦿ Для конкретного репозитория
- ⦿ Место хранения: `.git/config` репозитория
- ⦿ Команда: `git config`

Начальная настройка Git

 Необходимо задать два свойства

```
git config --global user.name "Ivan Ivanov"  
git config --global user.email ivan.ivanov@example.com
```

 Посмотреть значение свойства

```
git config user.name
```

 Посмотреть все настройки

```
git config --list --show-origin
```

 Встроенная подсказка по командам

```
git help <command>  
git <command> --help  
man git-<command>
```

Создание репозитория

- 📶 Перейдите в папку вашего проекта
- 📶 Выполните команду создания репозитория
 - 🕒 Директория `.git` будет создана по текущему пути

```
git init
```

- 📶 Если в папке проекта уже содержатся файлы, которые должны отслеживаться Git, добавьте их следующими командами

```
git add <file name>  
git add *.java  
git commit -m 'Initial project version'
```

- 📶 Теперь изменения в файлах отслеживаются Git

Клонирование репозитория

- 📶 Копирование всех данных удаленного репозитория в локальную систему
 - 🕒 Будут получены не только последние версии файлов, но и вся история их изменений

```
git clone <url>
```

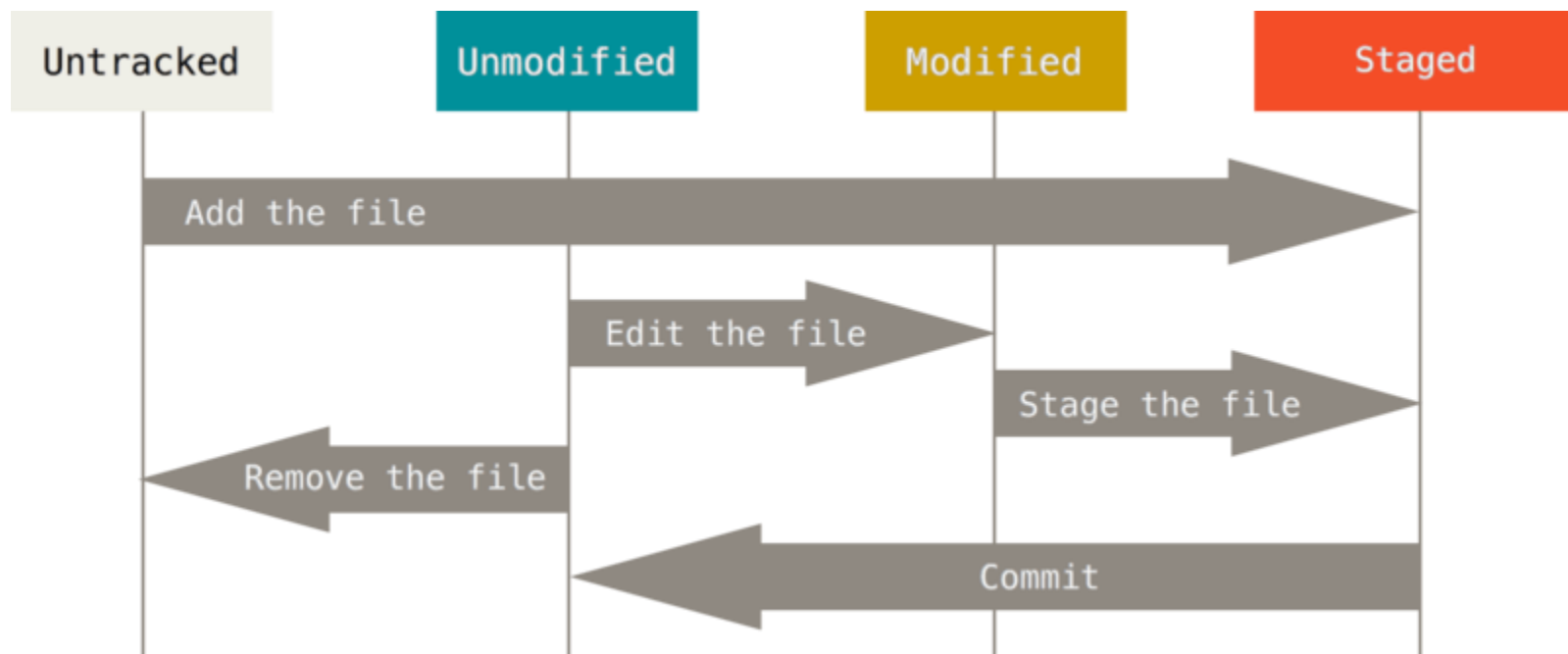
- 📶 Клонирование приводит к созданию локальной директории с именем клонируемого репозитория, в ней создается папка .git и разворачиваются файлы последней версии проекта
 - 🕒 Имя папки можно переопределить

```
git clone <url> <dir name>
```

```
git clone https://github.com/myAcc/myRepo myDirName
```

Запись изменений в репозиторий

- Для всех файлов, добавленных в Git, изменения отслеживаются автоматически
 - Если вы отредактируете файл и сохраните эти изменения Git пометит файл как измененный



Просмотр статуса файлов

- 🔊 Команда `status` позволяет увидеть состояние файлов в рабочей директории
 - ⦿ Не отслеживаемый
 - ⦿ Измененный, но не включенный в следующий коммит (not staged for commit)
 - ⦿ Измененный и включенный в коммит

```
git status [--short]
```

- 🔊 Команда `diff` показывает различия между предыдущим коммитом и текущим состоянием файлов

```
git diff
```

Для not staged изменений

```
git diff --staged
```

Для staged изменений

Добавление изменений и коммит

- 📡 Команда `add` добавляет изменения к следующему коммиту

```
git add <file name>
```

- 📡 Команда `commit` фиксирует все проиндексированные (staged) изменения как новую версию (коммит)

```
git commit -m <comment>
```

- 📡 Можно включить в коммит все изменения без их предварительной индексации
 - 🕒 Может привести к фиксации нежелательных изменений

```
git commit -a -m <comment>
```

Удаление и перемещение файлов

- 📶 Команда `rm` исключает файл из отслеживаемых и удаляет его из рабочей директории

```
git rm <file name>
```

- 📶 Если нужно исключить файл из отслеживаемых, но не удалять его из рабочей директории, следует использовать команду

```
git rm -cached <file name>
```

- 📶 Команда `mv` позволяет переименовывать файлы к рабочей директории, сохраняя их отслеживаемость

```
git mv <old file name> <new file name>
```

Игнорирование файлов

- 🌀 Если требуется, чтобы Git игнорировал определенные файлы в рабочей директории (не отслеживал их и не показывал в как неотслеживаемые) их включают в специальный список
 - 🌀 Обычно игнорируются автогенерируемые файлы
 - Журналы
 - Промежуточные файлы
 - Результаты сборки приложений
- 🌀 Файл `.gitignore` содержит список игнорируемых файлов
 - 🌀 Можно использовать Glob-шаблоны
 - 🌀 По умолчанию шаблон применяется рекурсивно ко всем поддиректориям
 - 🌀 `/` в начале шаблона запрещает рекурсию
 - 🌀 `/` в конце исключает каталог
 - 🌀 `!` в начале шаблона инвертирует его

Пример .gitignor

```
# Исключить все файлы с расширение .a  
*.a
```

```
# Но отслеживать файл lib.a даже если он подпадает  
# под исключение выше  
!lib.a
```

```
# Исключить файл TODO в корневой директории,  
# но не файл в subdir/TODO  
/TODO
```

```
# Игнорировать все файлы в директории build/  
build/
```

```
# Игнорировать файл doc/notes.txt, но не файл doc/server/arch.txt  
doc/*.txt
```

```
# Игнорировать все .txt файлы в директории doc/  
doc/**/*.txt
```

Работа с удаленными репозиториями

- 📶 Если вы создали локальный репозиторий клонированием удаленного, то связь между ними сохраняется автоматически
 - 🕒 По умолчанию удаленный сервер получает название origin
- 📶 Посмотреть список всех удаленных репозиторияев
 - 🕒 Можно увидеть их адреса для чтения и записи

```
git remote [-v]
```

- 📶 Добавление нового удаленного репозитория

```
git remote add <short name> <url>  
git remote add myrepo https://github.com/myAcc/myRepo
```

Отправка и получение изменений

- 📶 Команда `fetch` служит для получения всех данных проекта, которых нет в локальном репозитории

```
git fetch <short name>
```

- 📶 Команда `pull` служит для получения изменений из отслеживаемой ветки в удаленном репозитории и автоматического слияния с текущей локальной веткой

```
git pull origin master
```

- 📶 Команда `push` отправляет локальные изменения в глобальный репозитория

```
git push origin master
```

Ветвление

- 📶 Ветвление позволяет создавать альтернативные цепочки версий
 - 🕒 В командной разработке участники проекта могут работать над разными ветками
 - 🕒 Ветки помогают изолировать экспериментальный код от основного
 - 🕒 На основе веток можно создавать релизы для разных нужд
 - 🕒 Развитие продукта может идти по разным направлениям
- 📶 В git ветки – это подвижные указатели на коммиты
- 📶 Ветка **master** создается вместе с первым коммитом
 - 🕒 С каждым новым коммитом указатель **master** перемещается вперед
- 📶 Можно создавать новые ветки со своими именами, перемещать их указатели и переключаться между ними
- 📶 Специальный указатель **HEAD** указывает на текущую активную ветку

Перемещение указателя

- Команда `commit` создает новый коммит и перемещает указатель активной ветки на него

```
git commit -m 'New commit'
```



Создание новой ветки

🌀 Команда **branch** создает новую ветку

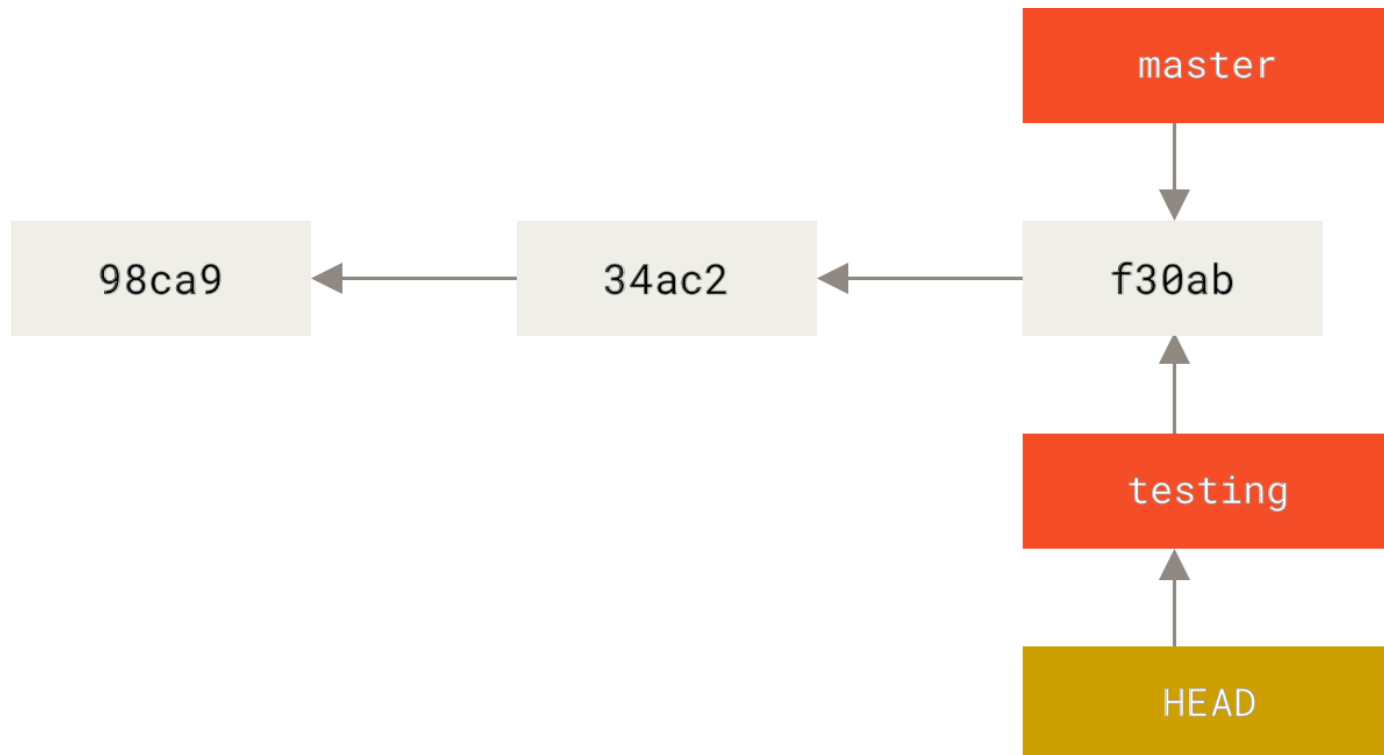
```
git branch testing
```



Переключение на новую ветку

- Команда `checkout` делает активной указанную ветку и перемещает на нее указатель **HEAD**

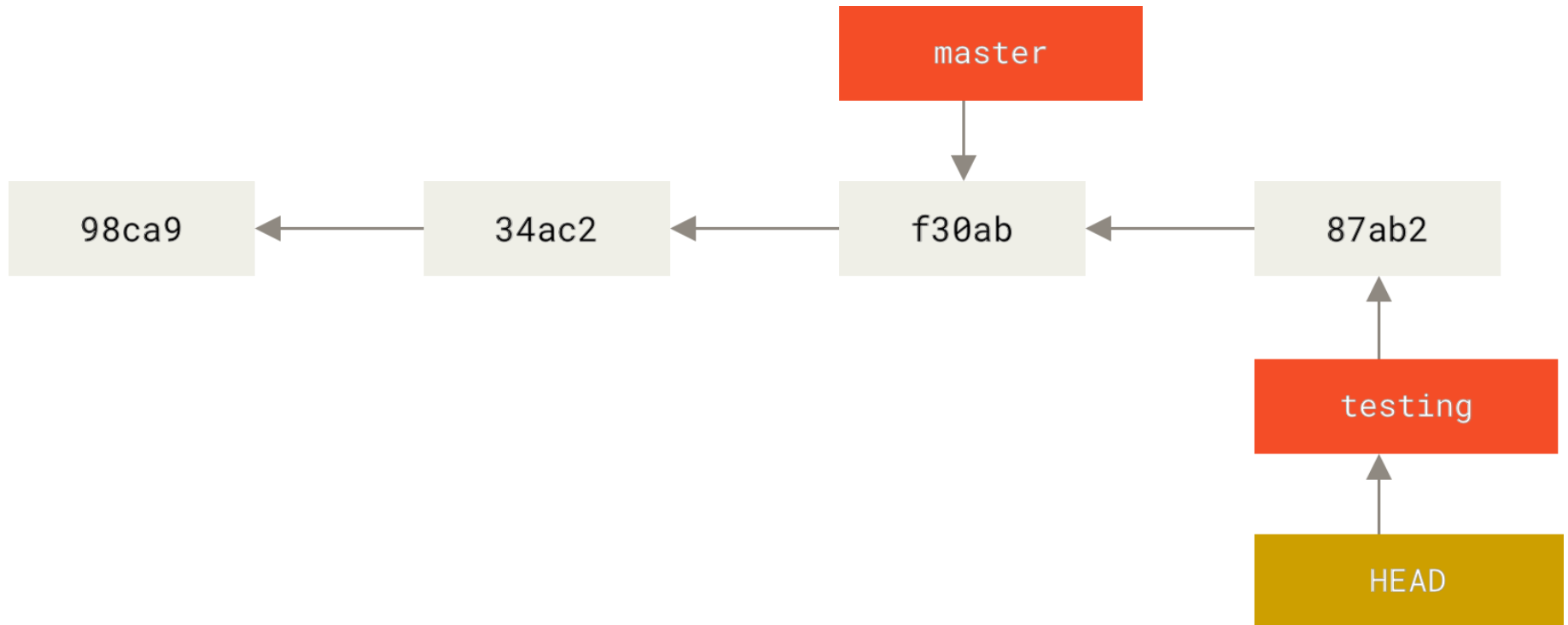
```
git checkout testing
```



Создание коммита в новой ветке

- Команда `commit` создает новый коммит в новой ветке и перемещает ее указатель

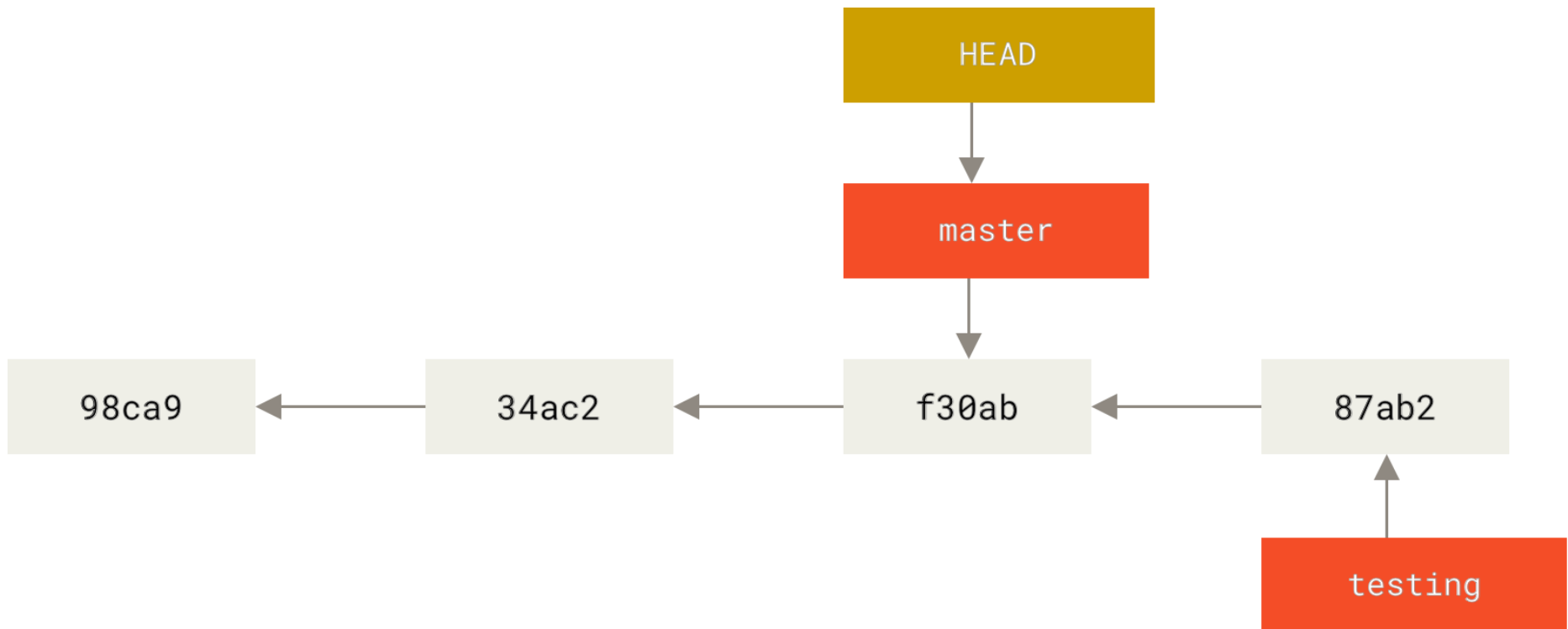
```
git commit -m 'Make a change'
```



Переключение на старую ветку

- Команда `checkout` делает активной старую ветку и перемещает на нее указатель **HEAD**

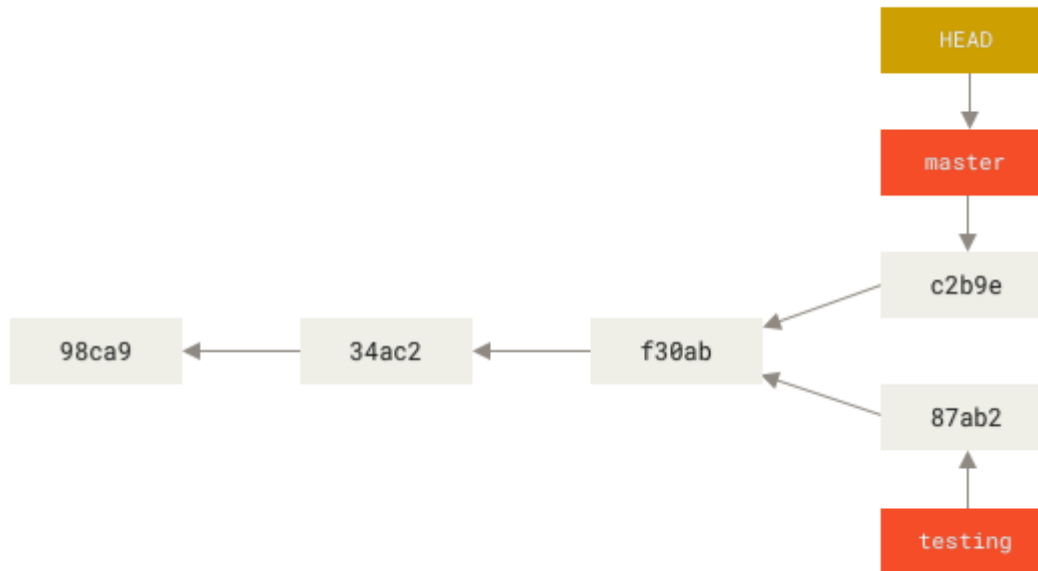
```
git checkout master
```



Создание альтернативной версии

- Команда `commit` создает новый коммит в старой ветке, теперь история версий проекта разделилась на две линии

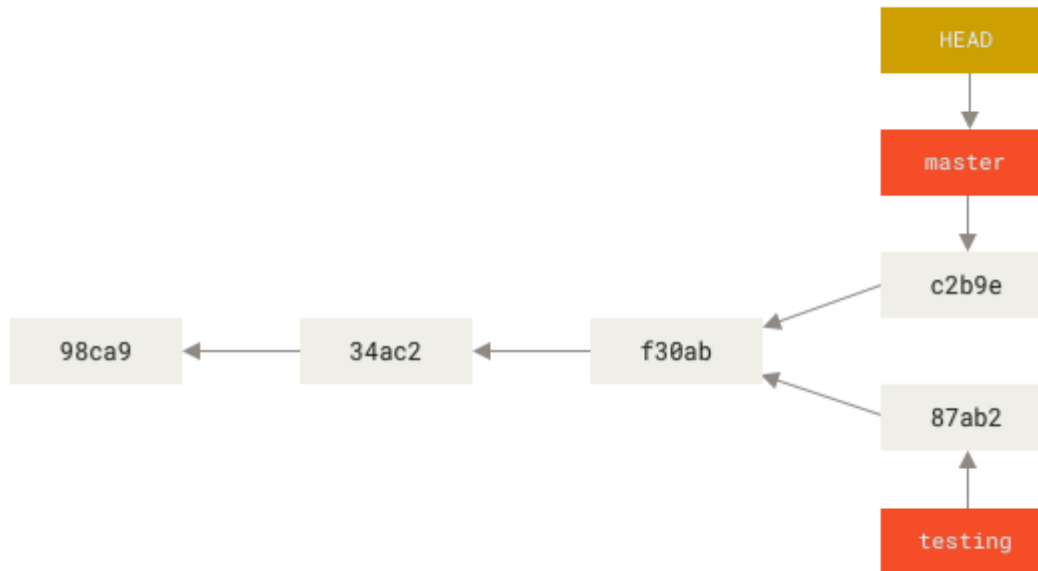
```
git commit -m 'Make other changes'
```



Создание альтернативной версии

- Команда `commit` создает новый коммит в старой ветке, теперь история версий проекта разделилась на две линии

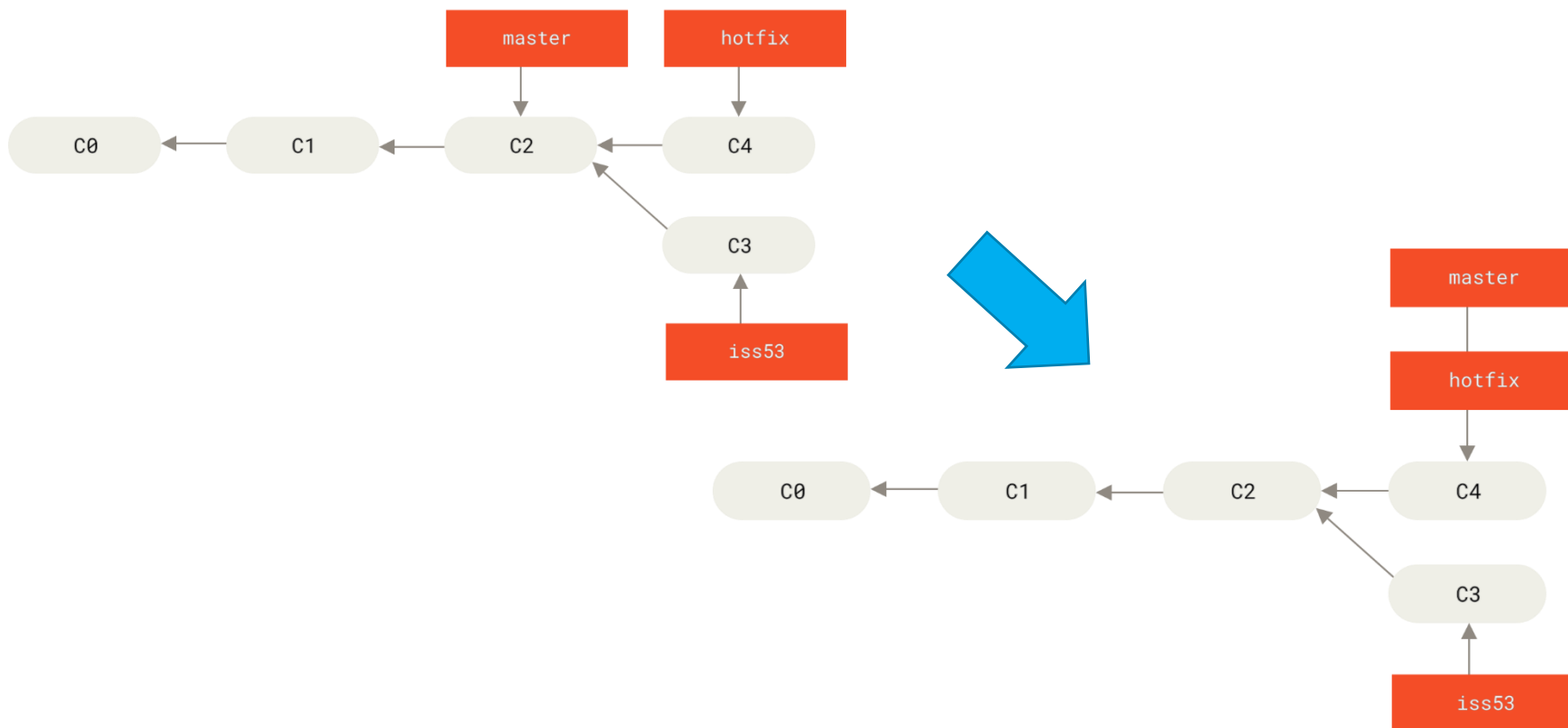
```
git commit -m 'Make other changes'
```



Объединение веток (merging)

 Команда `merge` объединяет ветки

```
git checkout master  
git merge hotfix
```



Удаление ветки

📶 Команда `branch -d` удаляет ветку

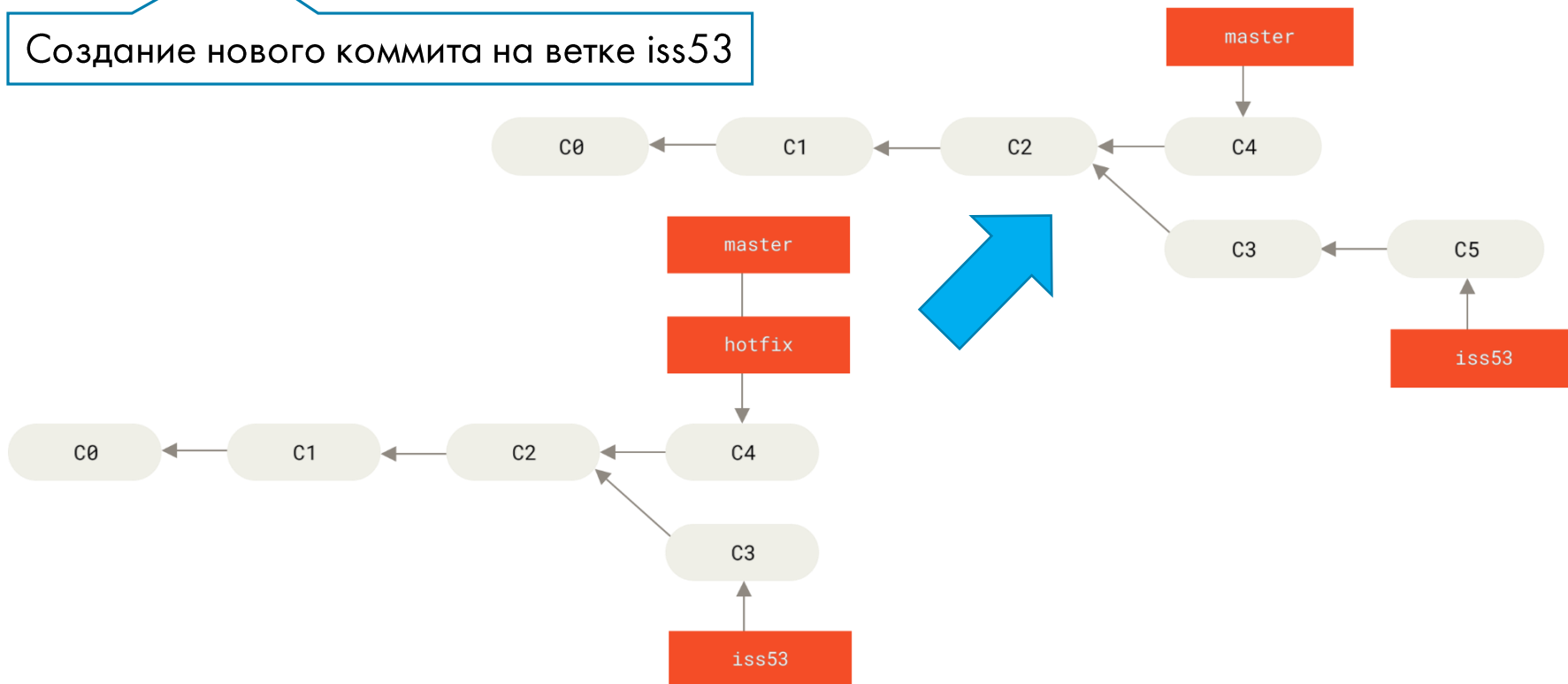
```
git branch -d hotfix
```

```
git checkout iss53
```

Переключение на ветку iss53

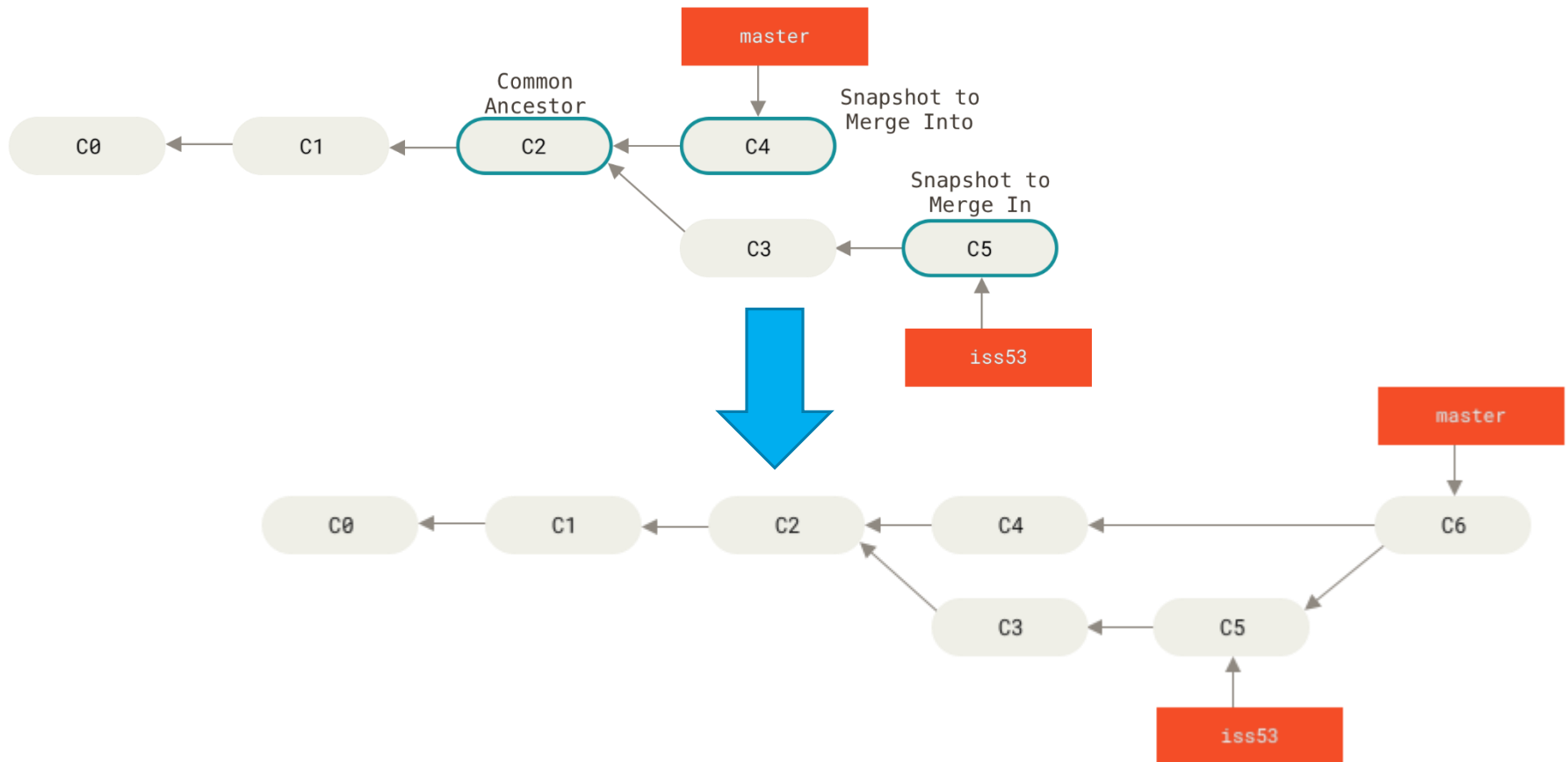
```
git commit -m 'Finish the new footer [issue 53]'
```

Создание нового коммита на ветке iss53



Объединение альтернативных версий

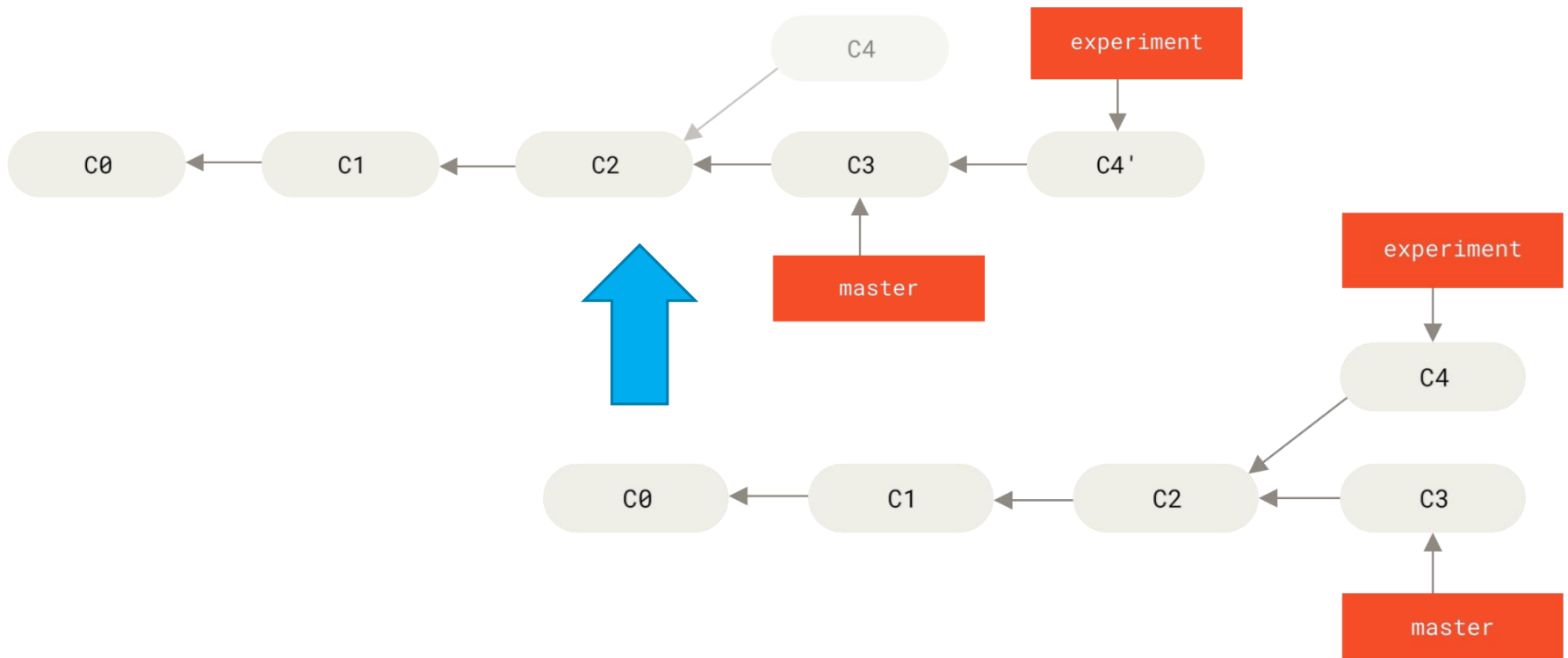
```
git checkout master  
git merge iss53
```



Смена родителя (rebasng)

🌀 Команда `rebase` объединяет ветки путем смены родителя

```
git checkout experiment  
git rebase master
```



Итоги

В данной лекции были рассмотрены следующие темы:

-  Стандартные и подключаемые библиотеки Python
-  Пакетные менеджеры
-  Системы управления версиями
-  Применение Git и командная разработка